Simple Portfolio Datenanalyse

Überblick

PROJECT1: CHILDHOOD ALLERGIES

- Project Overview & Goal
- Data Preprocessing & Visualization (**Python**)
  - Data Exploration
  - Data Preprocessing
  - Data Visualization
- Insight

PROJECT2: E-COMMERCE DATASET BY OLIST (SQLITE)

- Project Overview & Goal
- Data Preprocessing & Visualization
  - Data Exploration
  - Data Preprocessing (**SQL**)
  - Data Visualization (**Dashboard**)
- Insight

Project 1

CHILDHOOD ALLERGIES

Project Overview & Goal

Dieses Projekt nutzt den Datensatz „Childhood Allergies – Prevalence, Diagnosis and Treatment“, um die zehn am häufigsten vorkommenden Arten von Kinderallergien zu identifizieren. Außerdem soll analysiert werden, wie sich diese Allergien über Altersgruppen verteilen. Schließlich untersucht das Projekt, ob es altersabhängige Muster der Allergieanfälligkeit gibt.

- Dataset: Childhood Allergies: Prevalence, Demographics

Project Overview & Goal

Q1. Welche Allergien treten insgesamt am häufigsten auf?

Q2. Welche Allergien sind in welchen Altersgruppen häufiger?

Q3. Gibt es altersabhängige Muster der Allergieanfälligkeit?

Data Exploration

```python
import pandas as pd

# Data exploration
df = pd.read_csv('C:/Users/.../food_allergy.csv')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 333200 entries, 0 to 333199
Data columns (total 50 columns):
 #   Column                  Non-Null Count   Dtype
---  ------                  --------------   -----
 0   SUBJECT_ID              333200 non-null  int64
 1   BIRTH_YEAR              333200 non-null  int64
 2   GENDER_FACTOR           333200 non-null  object
 3   RACE_FACTOR             333200 non-null  object
 4   ETHNICITY_FACTOR        333200 non-null  object
 5   PAYER_FACTOR            333200 non-null  object
 6   ATOPIC_MARCH_COHORT     333200 non-null  bool
 7   AGE_START_YEARS         333200 non-null  float64
 8   AGE_END_YEARS           333200 non-null  float64
 9   SHELLFISH_ALG_START     5246 non-null    float64
 10  SHELLFISH_ALG_END       1051 non-null    float64
 11  FISH_ALG_START          1796 non-null    float64
 12  FISH_ALG_END            527 non-null     float64
 13  MILK_ALG_START          7289 non-null    float64
 14  MILK_ALG_END            4580 non-null    float64
 15  SOY_ALG_START           2419 non-null    float64
 16  SOY_ALG_END             1431 non-null    float64
 17  EGG_ALG_START           6065 non-null    float64
 18  EGG_ALG_END             3293 non-null    float64
 19  WHEAT_ALG_START         1146 non-null    float64
 20  WHEAT_ALG_END           688 non-null     float64
```

Data Preprocessing

1. Data Cleaning
2. Data Transformation
3. Data Integration
4. Data Reduction

```python
import pandas as pd

# Data exploration
...

# Data Preprocessing
# Q1. Which allergies are most common overall?

# 1. Data cleaning
# Age-based allergy distribution (only for patieents under 18)
df_child = df[df['AGE_START_YEARS'].notnull() & (df['AGE_START_YEARS'] < 18)].copy()

# The required variables sets
allergy_start_columns = [
    "SHELLFISH_ALG_START", "FISH_ALG_START", "MILK_ALG_START",
     ...
    "ALLERGIC_RHINITIS_START", "ASTHMA_START"
]

# 2. Data Transformation
# y-axis value: Number of patients
counts_allergy_start_child = df_child[allergy_start_columns].notna().sum().sort_values(ascending=False)

# Top 10 Allergies with the highest patient counts
top10_allergy_start = counts_allergy_start_child.head(10)

# Renaming variables for readability
allergy_new_name = {
    "SHELLFISH_ALG_START": "Shellfish Allergy",
    "FISH_ALG_START": "Fish Allergy",
     ...
    "ALLERGIC_RHINITIS_START": "Allergic Rhinitis",
    "ASTHMA_START": "Asthma"
}

# New variable names
top10_allergy_start_new_name = top10_allergy_start.rename(index=allergy_new_name)
```

Data Preprocessing

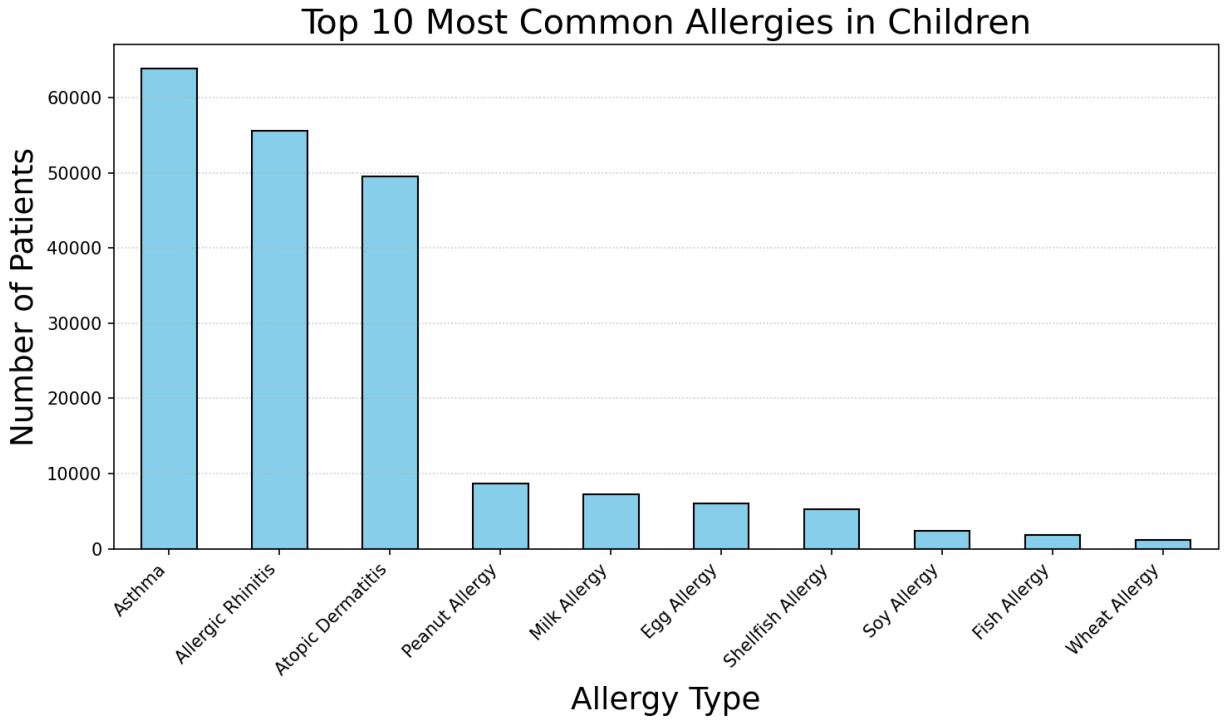
1. Data Cleaning
2. Data Transformation
3. Data Integration
4. Data Reduction

```python
# Q2. Which allergies are more common in which age groups?
# Data preprocessing

# 2. Data Transformation
# Define age groups (bins)
bins = [0, 5, 10, 15, 20]
labels = ['0-4', '5-9', '10-14', '15-19']
df_child['age_group'] = pd.cut(df_child['AGE_START_YEARS'], bins
    =bins, labels=labels, right=True)

# Extract the Top 10 allergy columns identified earlier
top10_columns = top10_allergy_start.index.tolist()

# Convert allergy indicators: boolean -> int
df_bin = df_child.copy()
df_bin[top10_columns] = df_bin[top10_columns].notnull().astype(
    int)

# 3. Data Reduction
# Aggregate total allergy counts for each age group
age_top10_sum = df_bin.groupby('age_group', observed=False)[
    top10_columns].sum()

# Map columns to more readable names
top10_new_name = {col: allergy_new_name[col] for col in
    top10_columns}
age_top10_sum_named = age_top10_sum.rename(columns=
    top10_new_name)

```

```python
# Q3. Are there age-related patterns in allergy susceptibility?
# Top 3 allergy columns & 'age_group'
top3_allergies = ['ASTHMA_START', 'ALLERGIC_RHINITIS_START', '
    ATOPIC_DERM_START', 'age_group']

# Create a new dataframe
df_pattern = df_child[top3_allergies].copy()

# Aggregate total allergy counts for each age group
age_grouped = df_pattern.groupby('age_group').sum()

# Reorder rows to match age group sequence
age_order = ['0-4', '5-9', '10-14', '15-19']
age_grouped = age_grouped.loc[age_order]
```

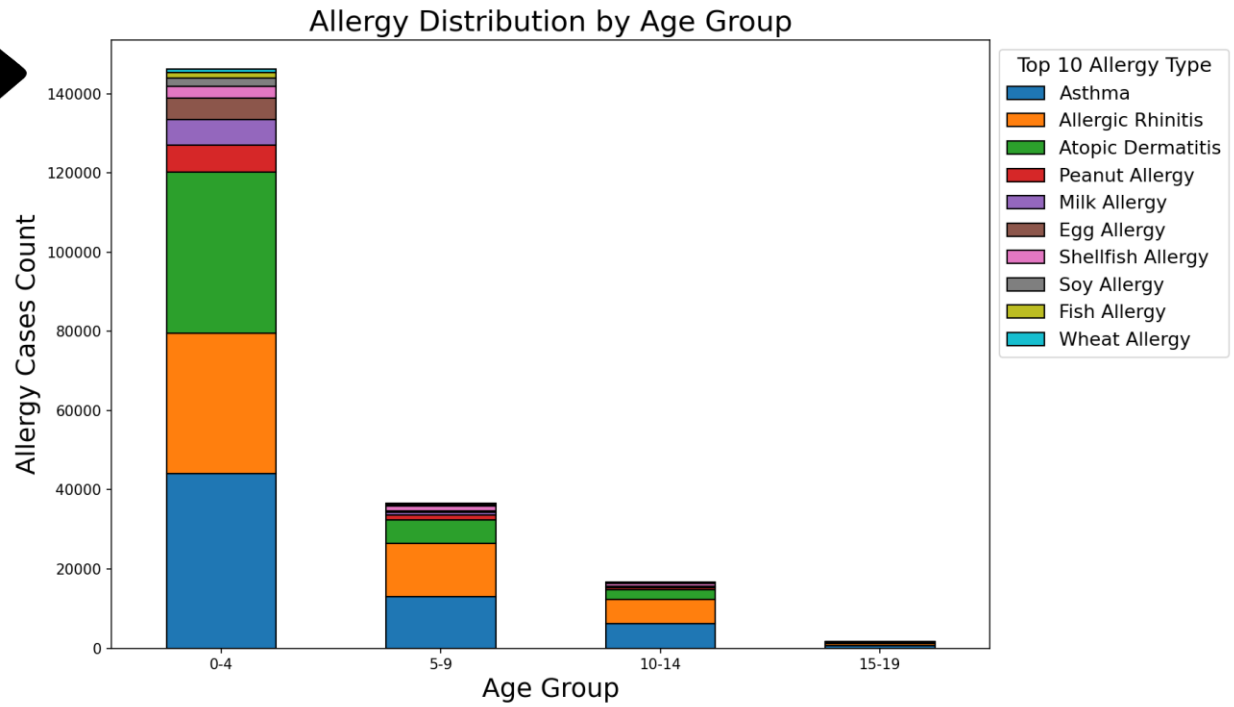
Visualization

```python
# Data Visualization for Q1
fig, ax = plt.subplots(figsize=(10,6))
top10_allergy_start_new_name.plot(kind='bar', color='skyblue',
    edgecolor='black', ax=ax)

ax.set_title('Top 10 Most Common Allergies in Children',
    fontsize=20)
ax.set_xlabel('Allergy Type', fontsize=18)
ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right'
    )
ax.set_ylabel('Number of Patients', fontsize=18)
ax.grid(axis='y', linestyle=':', alpha=0.7)

...
plt.tight_layout()
plt.savefig(fpath, dpi=150)
plt.close()
...
```



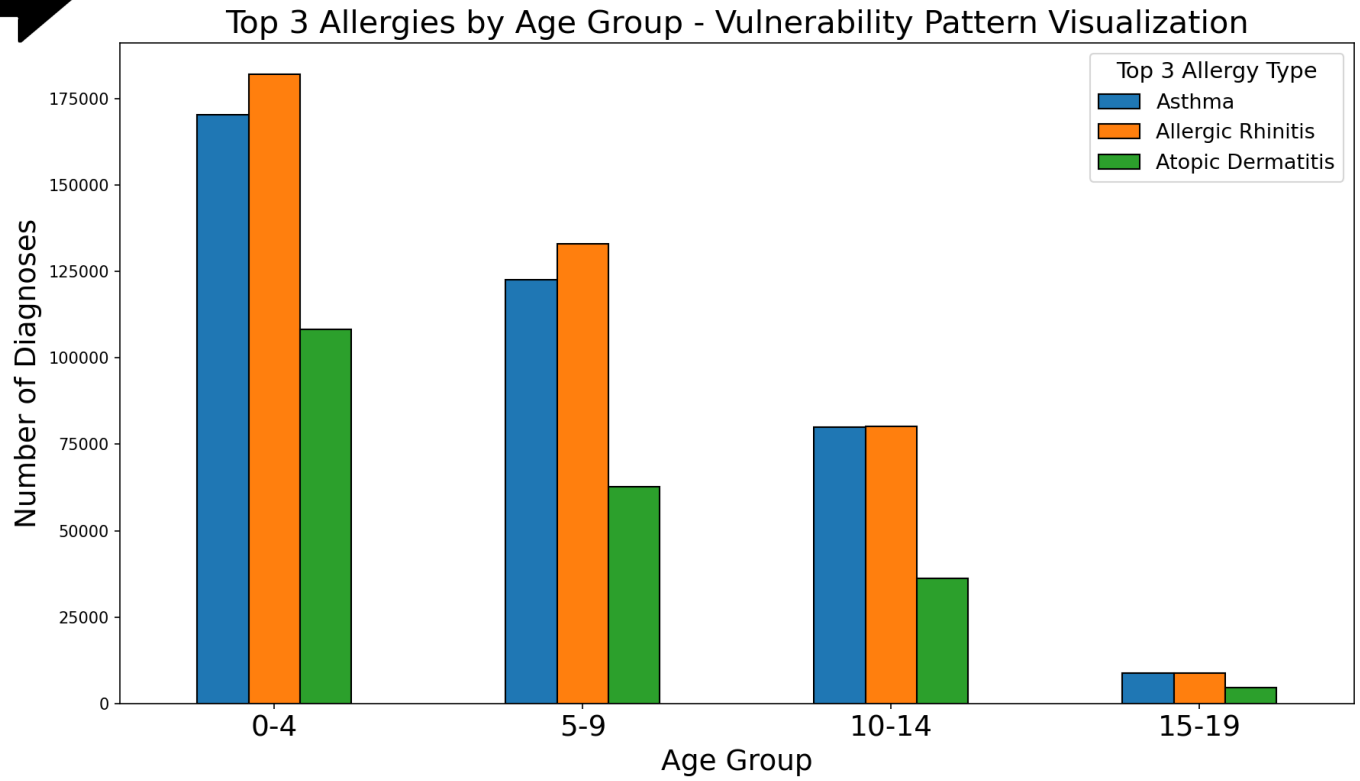
Visualization

```python
# Data Visualization for Q2
fig, ax = plt.subplots(figsize=(12,7))

# Generate visually distinct colors
colors = plt.cm.tab10(np.linspace(0, 1, len(top10_columns)))

age_top10_sum_named.plot(kind='bar', stacked=True, ax=ax,
    edgecolor='black', color=colors)

ax.set_title('Allergy Distribution by Age Group', fontsize=20)
ax.set_xlabel('Age Group', fontsize=18)
ax.set_xticklabels(ax.get_xticklabels(), rotation=0, ha='center'
    )
ax.set_ylabel('Allergy Cases Count', fontsize=18)
ax.legend(bbox_to_anchor=(1, 1), loc='upper left', title='Top 10
     Allergy Type', prop={'size':13}, title_fontsize=14)

...

```



Visualization

```python
# Data Visualization for Q3
fig, ax = plt.subplots(figsize=(12,7))
age_grouped.plot(kind='bar', ax=ax, edgecolor='black')

ax.set_title('Top 3 Allergies by Age Group - Vulnerability Pattern Visualization', fontsize=20)
ax.set_xlabel('Age Group', fontsize=18)
ax.set_xticks(range(len(age_order)))
ax.set_xticklabels(age_order, fontsize=18, rotation=0, ha='center')
ax.set_ylabel('Number of Diagnoses', fontsize=18)
ax.legend([allergy_new_name[col] for col in top3_allergies[:-1]], title='Top 3 Allergy Type')

...

```



Insight

- Insgesamt traten die drei häufigsten Allergien (Asthma, Allergische Rhinitis, Atopische Dermatitis) am häufigsten auf, wobei sich die relativen Anteile je nach Altersgruppe unterscheiden.
- In den Altersgruppen 0–4 und 5–9 war die allergische Rhinitis leicht häufiger als Asthma, während atopische Dermatitis niedriger lag.
- In den Altersgruppen 10–14 und 15–19 waren Asthma und allergische Rhinitis nahezu gleich, und atopische Dermatitis sank auf etwa die Hälfte.
- Mit zunehmendem Alter nahm die Gesamtprävalenz von Allergien tendenziell ab.
- Dies tendenziert das Vorhandensein altersabhängiger Anfälligkeitsmuster für Allergien.

Project 2

E-COMMERCE DATASET BY OLIST

Project Overview & Goal

In diesem Projekt wird das „E-commerce Dataset by Olist (SQLite)“ verwendet, um die Verteilung der Kundenbewertungen zu analysieren, die Anzahl verspäteter Lieferungen je Bewertungsgruppe zu ermitteln und schließlich zu untersuchen, ob Lieferverzögerungen einen Einfluss auf die abgegebene Bewertung haben.

- Dataset: E-commerce dataset by Olist (SQLite)

Project Overview & Goal

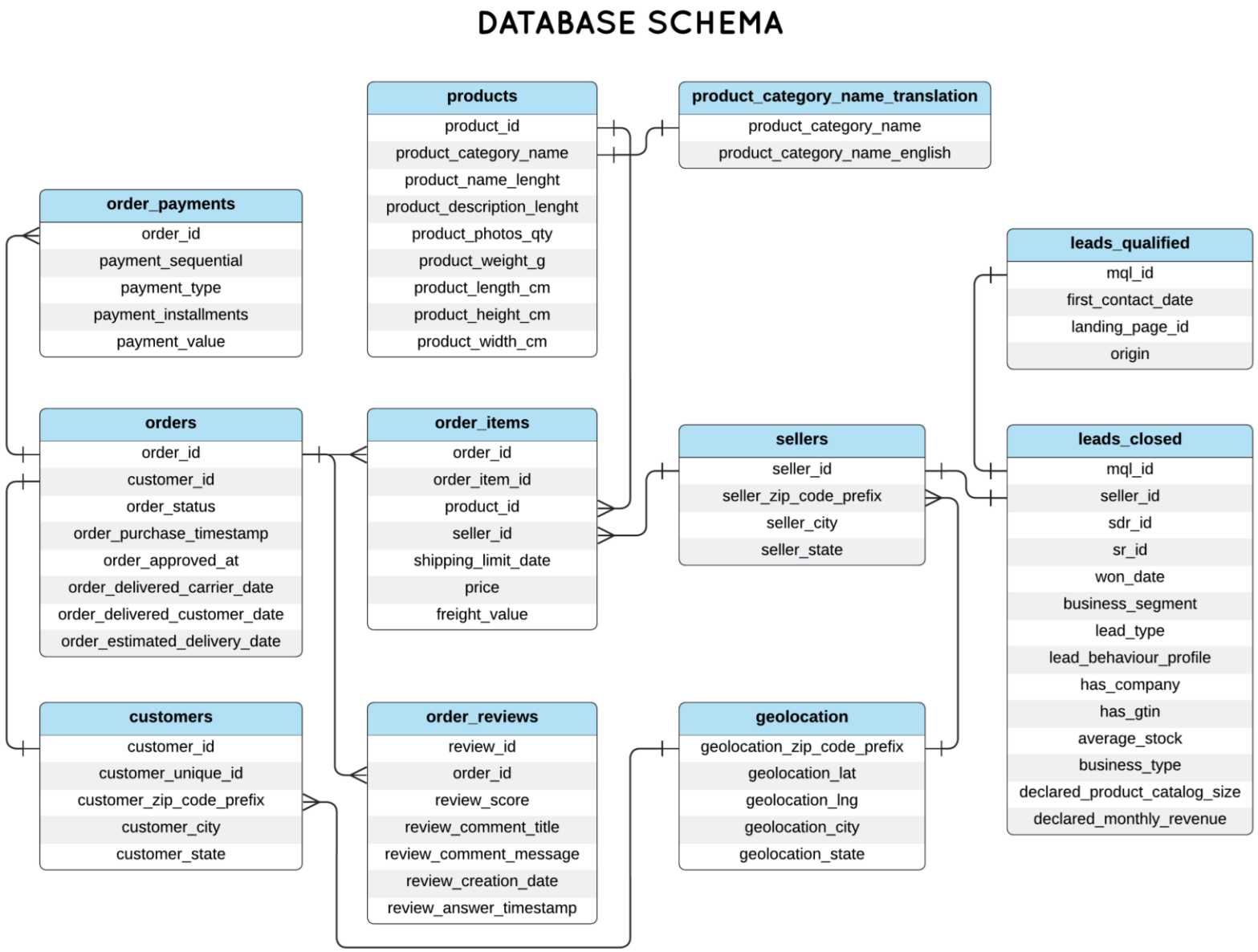
Q1. Wie verteilen sich die Rezensionen von 1 bis 5 Sternen?

Q2. Wie häufig treten Lieferverzögerungen auf?

Q3. Wie hoch ist die durchschnittliche Lieferverzögerung pro Bewertungsgruppe?

Data Exploration

Quelle: [E-commerce dataset by Olist (SQLite)]



Data Preprocessing

- Abfragen erstellen
  - Q1. Wie verteilen sich die Rezensionen von 1 bis 5 Sternen?

SQL File 4*

Limit to 1000 rows

```sql
#Q1. What is the distribution of review scores?
SELECT review_score, COUNT(*) AS cnt FROM order_reviews
    GROUP BY review_score
    ORDER BY review_score DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

review_score	cnt
5	57328
4	19142
3	8179
2	3151
1	11424

Data Preprocessing

- Abfragen erstellen
  - Q2. Wie häufig treten Lieferzögerungen auf?

SQL File 4*

Limit to 1000 rows

```sql
# Q2. What is the total number of delayed orders by review score?
SELECT review_score, COUNT(*) As delay_count FROM orders o
JOIN order_reviews r ON o.order_id = r.order_id
WHERE o.order_delivered_customer_date IS NOT NULL
AND o.order_delivered_customer_date > o.order_estimated_delivery_date
GROUP BY r.review_score
ORDER BY r.review_score DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

review_score	delay_count
5	1711
4	954
3	876
2	606
1	3554

Data Preprocessing

- Abfragen erstellen
  - Q3. Wie hoch ist die durchschnittliche Lieferverzögerung pro Bewertungsgruppen?

SQL File 4*

Limit to 1000 rows

```sql
# Q3. What is the average delivery delay by review score?
SELECT r.review_score, COUNT(*) AS total_reviews,
    SUM(CASE
            WHEN o.order_delivered_customer_date > o.order_estimated_delivery_date
            THEN 1
            ELSE 0
        END) AS delayed_count,
    AVG(CASE
            WHEN o.order_delivered_customer_date > o.order_estimated_delivery_date
            THEN DATEDIFF(o.order_delivered_customer_date, o.order_estimated_delivery_date)
            ELSE NULL
        END
    ) AS avg_delay_days
FROM order_reviews r
JOIN orders o ON r.order_id = o.order_id
GROUP BY r.review_score
ORDER BY r.review_score DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

review_score	total_reviews	delayed_count	avg_delay_days
5	57328	1711	4.3302
4	19142	954	5.7505
3	8179	876	7.2180
2	3151	606	9.3894
1	11424	3554	11.9778

Data Visualization

- Verbidung zum MySQL-Server herstellen
- Abfragen definieren
- SQL-Ausgaben in Dateframes umwandeln
- Dashboard-Struktur aufbauen
- KPI erstellen

```python
import streamlit as st
import pandas as pd
import plotly.express as px
from sqlalchemy import create_engine

# Connect to MySQL-Server
...

# Query for Q1, Q2 and Q3
query_q1 = """
SELECT review_score, COUNT(*) AS total
FROM order_reviews
GROUP BY review_score
ORDER BY review_score;
"""
...

# Convert SQL to dataframe
df_q1 = pd.read_sql(query_q1, engine)
df_q2 = pd.read_sql(query_q2, engine)
df_q3 = pd.read_sql(query_q3, engine)
df_reviews = pd.read_sql(query_q1, engine)
df_delayed = pd.read_sql(query_q2, engine)

# Dashboard
st.set_page_config(layout='wide')
st.markdown("""
    <style>
        .block-container {
            padding-top: 1rem;
            padding-bottom: 1rem;
        }
    </style>
""", unsafe_allow_html=True)
st.title('Olist E-commerce Review Dashboard')
st.write('Correlation Between Delivery Delays and Review Scores')

# KPI
st.subheader('Overview')

kpi_col1, kpi_col2 = st.columns(2)

with kpi_col1:
    st.metric('Total Reviews', df_reviews['total'].sum())

with kpi_col2:
    st.metric('Delayed Orders', df_delayed['delay_count'].sum())
```

Data Visualization

- Diagramme platzieren

```python
# Graphics
col1, col2, col3 = st.columns([1, 1, 1])

with col1:
    #st.header('Q1. What is the distribution of review ratings?')
    fig_q1_sort = df_q1.sort_values(by='review_score', ascending=False)

    fig_q1 = px.pie(fig_q1_sort, names='review_score', values='total',
                    title='Review Score Distribution',
                    color='review_score')
    fig_q1.update_layout(height=300, legend=dict(title='Review Score'))
    fig_q1.update_traces(sort=False)
    fig_q1.update_layout(legend=dict(traceorder='normal')
)
    st.plotly_chart(fig_q1, use_container_width=True)

with col2:
    #st.header('Q2. Count delayed cases by each review score')
    fig2 = px.bar(df_q2, x='review_score', y='delay_count',
                  labels={'review_score': 'Review Score', 'delay_count': 'Delayed Orders'},
                  title='Count of Delayed Orders by Review Score',
                  color='delay_count')
    fig2.update_layout(height=300)
    st.plotly_chart(fig2, use_container_width=True)

with col3:
    #st.header('Q3. Delayed Days by each review score')
    fig3 = px.line(df_q3, x='review_score', y='avg_delay_days',
                   markers=True,
                   labels={'review_score': 'Review Score', 'avg_delay_days': 'Average Delay (Days)'},
                   title='Average Delivery Delay by Review Score')
    fig3.update_layout(height=300)
    st.plotly_chart(fig3, use_container_width=True)
```

Data Visualization (Dashboard)

Olist E-commerce Review Dashboard

Correlation Between Delivery Delays and Review Scores

Overview

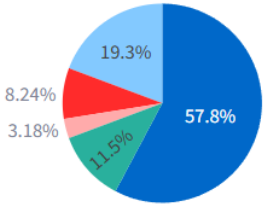
Total Reviews

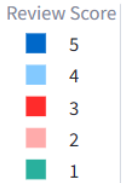
99224

Delayed Orders

7701

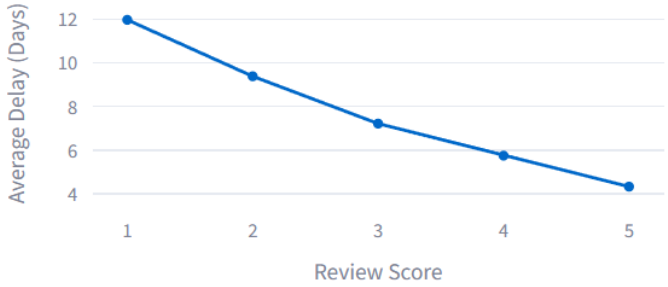
Review Score Distribution





Count of Delayed Orders by Review Score

Average Delivery Delay by Review Score



Insight

- Über 85 % der Bewertungen liegen bei mindestens 3 Sternen, was auf eine hohe generelle Zufriedenheit hinweist.
- Die meisten Lieferverzögerungen treten bei 1-Sterne-Bewertungen auf. Trotz ihrer hohen Gesamtzahl zeigt sich auch bei 5-Sterne-Bewertungen ein relevanter Anteil verspäteter Lieferungen.
- Die durchschnittliche Lieferverzögerung beträgt ca. 4 Tage bei 5 Sternen, aber rund 12 Tage bei 1 Stern.
- Insgesamt lässt sich erkennen: Lange Lieferverzögerungen erhöhen deutlich die Wahrscheinlichkeit negativer Bewertungen.

Reference

1. Childhood Allergies: Prevalence, Demographics
2. E-commerce dataset by Olist (SQLite)
3. API reference
4. MySQL Tutorial
5. 14.7 Date and Time Functions
6. MySQL 8.4 Reference Manual
7. Streamlit API reference - st.markdown

Thank you.